

Lesson 1 — Why Sparse Matrices Exist

1. Where We Are in the Curriculum

You are starting **Unit A: Sparse Matrix Foundations**.

This unit teaches the basic ideas behind sparse matrices before writing full programs.

Current progress

Lesson	Topic	Status
Lesson 1	Why sparse matrices exist	Current lesson
Lesson 2	Coordinate List representation and creation	Not started
Lesson 3	Complete C sparse matrix display program	Not started

After this lesson, you should be able to explain:

1. What a matrix is.
2. What a sparse matrix is.
3. Why sparse matrices save memory.
4. How Coordinate List representation works.
5. How CSR representation works.
6. Why `IA` has `rows + 1` entries in CSR.
7. Why sparse matrix space can be better than normal matrix space.

2. What Is a Matrix?

A **matrix** is a table of numbers arranged in rows and columns.

Example:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

This matrix has:

```
3 rows
4 columns
```

So the total number of positions is:

```
3 × 4 = 12 elements
```

That means this matrix has 12 places where values can be stored.

3. What Is a Sparse Matrix?

A **sparse matrix** is a matrix where most of the values are zero.

Example:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

Most of the values are `0`.

Only two values are useful:

```
5 at row 0, column 3
7 at row 2, column 1
```

So instead of storing every value, we can store only the values that are not zero.

This is the main idea of sparse matrices.

4. Why Do Sparse Matrices Matter?

A normal matrix stores every value, including zeros.

For this matrix:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

A normal 2D array stores all 12 elements:

```
0, 0, 0, 5, 0, 0, 0, 0, 0, 7, 0, 0
```

But most of these values are zero.

A sparse representation stores only this:

row	column	value
0	3	5
2	1	7

This saves memory because we do not store useless zeros.

Sparse matrices are especially useful when the matrix is very large but has only a few non-zero values.

Main benefits

Sparse matrices help save:

- Memory
- Processing time

5. Important Variables

In sparse matrix programs, these variable names are commonly used:

```
m   = number of rows
n   = number of columns
num = number of non-zero elements
```

Example matrix:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

Here:

```
m = 3
n = 4
num = 2
```

Why?

- There are 3 rows.
- There are 4 columns.
- There are 2 non-zero values: 5 and 7.

Important: `num` does **not** mean the total number of matrix positions.

It means only the number of values that are not zero.

6. Normal Matrix Storage

In normal matrix storage, every element is stored.

Example:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

This matrix has 12 elements, so a normal array stores all 12 values.

Even the zeros are stored.

That is wasteful when most values are zero.

Normal matrix space complexity

If a matrix has:

```
m rows
n columns
```

Then total elements are:

```
m × n
```

So the space complexity is:

```
O(m × n)
```

This means memory usage grows based on the full size of the matrix.

7. Sparse Matrix Storage Idea

Sparse matrix storage uses a different idea:

Instead of storing all values, store only non-zero values.

For each non-zero value, we need to know:

1. Which row it is in.
2. Which column it is in.
3. What the value is.

Example:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

Only two values are stored:

row	column	value
0	3	5
2	1	7

8. Coordinate List Representation

The simplest sparse matrix representation is called **Coordinate List**.

It is also called **COO**.

COO stores each non-zero element using three values:

```
row number
column number
value
```

Example matrix:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

Coordinate List form:

row	column	value
0	3	5
2	1	7

This means:

At position (0, 3), the value is 5.
At position (2, 1), the value is 7.

9. Coordinate List with a First Row

Sometimes, in theory, Coordinate List includes a special first row.

This first row stores general information about the matrix:

```
total rows, total columns, number of non-zero values
```

For this matrix:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

The matrix has:

```
3 rows
4 columns
2 non-zero values
```

So the Coordinate List can be written as:

row	column	value
3	4	2
0	3	5
2	1	7

The first row:

```
3 4 2
```

means:

```
3 rows, 4 columns, 2 non-zero values
```

The remaining rows store the actual non-zero elements.

10. How This Appears in C/C++ Code

In actual C or C++ code, we usually do not store the first row inside the element array.

Instead, we store matrix information separately.

For example:

```
m    // total rows
n    // total columns
num  // number of non-zero elements
```

Then we store only the non-zero elements in an array.

Each non-zero element needs:

```
row
column
value
```

So the structure idea is:

```
struct Element {
    int i; // row number
    int j; // column number
    int x; // value
};
```

And the sparse matrix stores:

```
struct Sparse {
    int m;           // total rows
    int n;           // total columns
    int num;         // number of non-zero elements
    struct Element *e; // array of non-zero elements
};
```

You will use this idea more in Lesson 2.

11. CSR Representation

CSR means **Compressed Sparse Row**.

CSR is another way to store a sparse matrix.

It is more compact than Coordinate List in many cases because it does not store the row number with every single non-zero value.

CSR uses three arrays:

```
A = stores non-zero values
IA = stores row pointer information
JA = stores column indexes
```

12. Meaning of CSR Arrays

A array

A stores the actual non-zero values.

Example:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

The non-zero values are:

5, 7

So:

A = [5, 7]

JA array

JA stores the column index of each value in **A**.

For the same matrix:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

Value 5 is in column 3.

Value 7 is in column 1.

So:

JA = [3, 1]

IA array

IA stores row pointer information.

It tells us where each row starts and ends inside the A array.

This is the most important part of CSR.

13. Building the CSR Arrays Step by Step

Use this matrix:

```
0  0  0  5
0  0  0  0
0  7  0  0
```

There are 3 rows.

There are 2 non-zero values.

First, store the non-zero values:

A = [5, 7]

Then store their column indexes:

JA = [3, 1]

Now build IA.

Start with:

$IA[0] = 0$

This means before row 0 begins, we have stored 0 non-zero values.

Row 0

Row 0 is:

0 0 0 5

There is 1 non-zero value.

The cumulative count becomes 1.

So:

$IA[1] = 1$

Row 1

Row 1 is:

0 0 0 0

There are no non-zero values.

The cumulative count stays 1.

So:

$IA[2] = 1$

Row 2

Row 2 is:

```
0  7  0  0
```

There is 1 non-zero value.

The cumulative count becomes 2.

So:

```
IA[3] = 2
```

Final CSR form

```
A  = [5, 7]
JA = [3, 1]
IA = [0, 1, 1, 2]
```

14. Why Does IA Have `rows + 1` Entries?

This is a key CSR idea.

If a matrix has `m` rows, then `IA` has:

```
m + 1 entries
```

For example, if the matrix has 3 rows:

```
IA size = 3 + 1 = 4
```

Why do we need one extra value?

Because for each row, CSR uses this formula:

```
start = IA[row]
end   = IA[row + 1]
```

The extra value lets us find the end of the last row.

15. Using IA to Find Row Values

Given:

```
A  = [5, 7]
JA = [3, 1]
IA = [0, 1, 1, 2]
```

Row 0

```
start = IA[0] = 0
end   = IA[1] = 1
```

So row 0 uses values from index 0 up to before index 1.

That means row 0 contains:

```
A[0] = 5
```

Its column index is:

```
JA[0] = 3
```

So row 0 has:

```
value 5 at column 3
```

Row 1

```
start = IA[1] = 1  
end   = IA[2] = 1
```

Start and end are the same.

That means row 1 has no non-zero values.

Row 2

```
start = IA[2] = 1  
end   = IA[3] = 2
```

So row 2 uses:

```
A[1] = 7
```

Its column index is:

```
JA[1] = 1
```

So row 2 has:

```
value 7 at column 1
```

16. Coordinate List vs CSR

Both Coordinate List and CSR store only non-zero values.

But they organize the information differently.

Representation	What it stores	Main idea
Coordinate List	Row, column, value for each non-zero element	Simple and easy to understand

Representation	What it stores	Main idea
CSR	Values, column indexes, row pointers	More compact for row-based access

Coordinate List example

row	column	value
0	3	5
2	1	7

CSR example

```
A = [5, 7]
JA = [3, 1]
IA = [0, 1, 1, 2]
```

Coordinate List stores a row number for every non-zero value.

CSR avoids storing every row number separately.

Instead, CSR uses **IA** to describe row positions.

17. Memory Comparison Example

Suppose we have an **8 × 9** matrix.

A normal matrix stores:

$8 \times 9 = 72$ values

If each integer takes 2 bytes:

$72 \times 2 = 144$ bytes

So the normal matrix needs 144 bytes.

CSR storage example

The CSR example uses:

A = 8 values
IA = 9 values
JA = 8 values

Total integer slots:

$8 + 9 + 8 = 25$ slots

If each integer takes 2 bytes:

$25 \times 2 = 50$ bytes

So:

Normal matrix storage = 144 bytes
CSR storage = 50 bytes

CSR uses much less memory in this example.

18. Space Complexity

Normal matrix

A normal matrix stores all positions.

If there are m rows and n columns:

Total positions = $m \times n$

So normal matrix space is:

$O(m \times n)$

CSR matrix

CSR stores:

```
A = num values
JA = num column indexes
IA = m + 1 row pointers
```

So total storage is about:

```
num + num + (m + 1)
```

In Big-O notation, constants are ignored.

So CSR space is written as:

```
O(num + m)
```

This means CSR storage depends mainly on:

```
number of non-zero values + number of rows
```

It does not depend directly on the full $m \times n$ size.

That is why CSR is useful when `num` is much smaller than $m \times n$.

19. Important Lesson 1 Vocabulary

Term	Simple meaning
Matrix	A table of numbers arranged in rows and columns
Sparse matrix	A matrix where most values are zero
Non-zero element	A value in the matrix that is not zero
Row	A horizontal line of values
Column	A vertical line of values
<code>m</code>	Number of rows

Term	Simple meaning
<code>n</code>	Number of columns
<code>num</code>	Number of non-zero elements
Coordinate List / COO	Stores row, column, and value for each non-zero element
CSR	Compressed Sparse Row representation
<code>A</code>	CSR array that stores non-zero values
<code>JA</code>	CSR array that stores column indexes
<code>IA</code>	CSR array that stores row pointer information
<code>O(m × n)</code>	Space needed by normal matrix storage
<code>O(num + m)</code>	Space needed by CSR storage

20. Common Beginner Mistakes

Mistake 1: Thinking `num` means total matrix size

Wrong:

```
num = m × n
```

Correct:

```
num = number of non-zero values only
```

Example:

```
0 0 0 5
0 0 0 0
0 7 0 0
```

Here:

```
m × n = 3 × 4 = 12
num = 2
```

Mistake 2: Forgetting that IA size is rows + 1

If there are 3 rows, `IA` does not have 3 values.

It has:

```
3 + 1 = 4 values
```

The extra value helps us find the end of the last row.

Mistake 3: Mixing up A and JA

Remember:

```
A  stores actual values
JA stores column indexes
```

Example:

```
A  = [5, 7]
JA = [3, 1]
```

This means:

```
5 is in column 3
7 is in column 1
```

Mistake 4: Thinking CSR stores row numbers directly

Coordinate List stores row numbers directly.

CSR does not store row numbers for every value.

CSR uses `IA` to know which row each value belongs to.

21. Lesson 1 Summary

A sparse matrix is a matrix with mostly zero values.

Normal matrix storage keeps every value, including zeros.

Sparse matrix storage keeps only non-zero values.

Coordinate List stores each non-zero value like this:

```
row, column, value
```

CSR stores sparse matrices using three arrays:

```
A = non-zero values  
JA = column indexes  
IA = row pointer array
```

Normal matrix space:

```
O(m × n)
```

CSR space:

```
O(num + m)
```

where:

```
m = number of rows  
n = number of columns  
num = number of non-zero elements
```

The big idea is simple:

Do not waste memory storing zeros when most of the matrix is empty.

22. Practice Question

Convert this matrix into Coordinate List form:

```
0 4 0
0 0 0
7 0 9
```

First find:

```
m = ?
n = ?
num = ?
```

Then write each non-zero value as:

```
row    column    value
```

23. Practice Answer

The matrix is:

```
0 4 0
0 0 0
7 0 9
```

It has:

```
m = 3
n = 3
num = 3
```

The non-zero values are:

```
4 at row 0, column 1
7 at row 2, column 0
9 at row 2, column 2
```

Coordinate List:

row	column	value
0	1	4
2	0	7
2	2	9

With the theoretical first row included:

row	column	value
3	3	3
0	1	4
2	0	7
2	2	9

24. Final Check Before Lesson 2

Before moving to Lesson 2, make sure you can answer these:

1. What is a sparse matrix?
2. Why is storing zeros wasteful?
3. What does `num` mean?
4. What three things does Coordinate List store?
5. What are the three CSR arrays?
6. Why does `IA` have `rows + 1` entries?
7. Why is normal matrix space $O(m \times n)$?
8. Why is CSR space $O(\text{num} + m)$?

If you can answer these, you are ready for Lesson 2.